BCAC601 - Advance Java with Web Application by Anik

M1 – Introduction to Java EE

Introduction to Java EE: Overview of Java EE Architecture

Introduction

Java EE (Java Enterprise Edition), now known as Jakarta EE, is a platform built on top of Java SE (Standard Edition) for developing large-scale, distributed, secure, and enterprise-level applications. It provides a set of APIs, specifications, and services that simplify the development of web-based and multi-tier applications.

Java EE enables developers to build applications that can handle multiple users simultaneously, manage transactions, interact with databases, and provide security features. It eliminates the need to write low-level code by providing reusable components and services.

Enterprise applications developed using Java EE are portable, scalable, and platform-independent. They can run on any operating system that supports a Java EE-compatible application server.

Features of Java EE

1. Platform Independence – Applications can run on different operating systems without modification.
2. Scalability – Supports applications that can grow from a small number of users to millions of users.
3. Multi-tier Architecture – Separates application logic into different layers for better maintainability.
4. Web Support – Provides technologies such as Servlets, JSP, and JSF for web application development.
5. Database Connectivity – Supports interaction with databases through JDBC and JPA.
6. Security – Provides authentication, authorization, and role-based access control.
7. Transaction Management – Ensures data consistency during database operations.
8. Distributed Computing – Allows applications to run across multiple servers.
9. Reusability – Components can be reused in different applications.
10. Automatic Resource Management – Application servers manage resources such as database connections and threads.

Java EE Architecture

Java EE follows a multi-tier architecture that divides an application into separate layers. Each layer performs a specific function and communicates with other layers.

1. Client Tier

The Client Tier is the front-end layer that interacts directly with users.

Components:

- Web Browsers
- Desktop Applications
- Mobile Applications

Functions:

- Accepts user input.
- Sends requests to the server.
- Displays the results returned by the server.

Examples:

- Google Chrome
- Mozilla Firefox
- Mobile Apps

2. Web Tier

The Web Tier processes client requests and generates responses.

Components:

- Servlets
- JSP (Java Server Pages)
- JSF (JavaServer Faces)

Functions:

- Receives HTTP requests from clients.
- Processes user input.
- Generates dynamic web pages.
- Communicates with the Business Tier.

Advantages:

- Separates presentation logic from business logic.
- Supports dynamic content generation.

3. Business Tier

The Business Tier contains the business logic of the application.

Components:

- Enterprise JavaBeans (EJB)
- Business Services

- Session Beans

Functions:

- Processes business rules.
- Handles calculations and validations.
- Manages transactions.
- Communicates with the database layer.

Advantages:

- Centralized business processing.
- Easy maintenance and scalability.

4. Enterprise Information System (EIS) Tier

The EIS Tier stores and manages enterprise data.

Components:

- Relational Databases
- Legacy Systems
- ERP Systems
- CRM Systems

Functions:

- Stores application data.
- Performs data retrieval and updates.
- Maintains data integrity.

Examples:

- MySQL
- Oracle Database
- PostgreSQL
- Microsoft SQL Server

Working of Java EE Architecture

Step 1:
A user sends a request through a web browser.

Step 2:
The request reaches the Web Tier, where Servlets or JSP pages process it.

Step 3:
The Web Tier forwards the request to the Business Tier.

Step 4:
The Business Tier executes business logic and validates the request.

Step 5:
If data is required, the Business Tier communicates with the EIS Tier.

Step 6:
The database processes the query and returns the result.

Step 7:
The Business Tier processes the result and sends it to the Web Tier.

Step 8:
The Web Tier generates the final response and sends it back to the client.

Diagram

Java EE Multi-Tier Architecture

Client Tier → Web Tier → Business Tier → EIS Tier (Database)

Example:

Browser → Servlet/JSP → EJB/Business Logic → MySQL/Oracle Database

Advantages of Java EE Architecture

1. Better organization of application components.
2. Easy maintenance and modification.
3. High scalability for enterprise applications.
4. Improved security mechanisms.
5. Efficient transaction management.
6. Reusable business components.
7. Platform-independent deployment.
8. Simplified database integration.

Applications of Java EE

1. Banking Systems
2. E-Commerce Applications
3. Online Reservation Systems
4. Enterprise Resource Planning (ERP)
5. Customer Relationship Management (CRM)
6. Hospital Management Systems
7. Government Portals
8. Educational Management Systems

Difference Between Java SE and Java EE

Subject	Java SE	Java EE
Full Form	Java Standard Edition	Java Enterprise Edition
Purpose	Used for standalone and desktop applications	Used for enterprise and web applications
Application Type	Small and medium-sized applications	Large-scale enterprise applications
APIs	Provides core Java APIs	Provides enterprise APIs along with Java SE APIs
Technologies Used	AWT, Swing, JavaFX	Servlets, JSP, EJB, Web Services
Web Support	No built-in web support	Complete web application support
Architecture	Simple architecture	Multi-tier architecture
Execution Environment	Runs directly on JVM	Runs on application servers
Scalability	Limited scalability	Highly scalable
Transaction Management	Not provided	Provided
Security Features	Basic security	Advanced enterprise security
Examples	Calculator, Text Editor, Desktop Applications	Banking Systems, ERP, E-Commerce Websites

Role of JDBC

Introduction

JDBC (Java Database Connectivity) is a standard Java API that enables Java applications to communicate with databases. It acts as a bridge between a Java program and a database management system (DBMS). JDBC allows developers to execute SQL queries, retrieve data, update records, and manage database connections directly from Java applications.

Roles of JDBC

1. **Establishes Database Connection**
    - JDBC provides methods to connect Java applications with various databases such as MySQL, Oracle, PostgreSQL, and SQL Server.
2. **Executes SQL Statements**
    - Allows execution of SQL commands such as SELECT, INSERT, UPDATE, DELETE, CREATE, and DROP.
3. **Retrieves Data**
    - Fetches data from the database and stores the result in a ResultSet object for processing.
4. **Updates Database Records**
    - Enables modification, insertion, and deletion of records in database tables.
5. **Provides Database Independence**

    - The same JDBC code can work with different databases by changing the JDBC driver.

6. **Supports Transaction Management**
    - Ensures data consistency through COMMIT and ROLLBACK operations.
7. **Handles Exceptions**
    - Provides exception handling mechanisms to manage database errors effectively.
8. **Improves Security**
    - Supports PreparedStatement, which helps prevent SQL Injection attacks.
9. **Facilitates Enterprise Application Development**
    - JDBC is widely used in web applications, enterprise systems, and distributed applications for database interaction.

JDBC Architecture Components

- JDBC API
- JDBC Driver Manager
- JDBC Driver
- Database Connection
- Database Management System (DBMS)

Applications of JDBC

- Banking Systems
- E-Commerce Applications
- Student Management Systems
- Inventory Management Systems
- Hospital Management Systems
- Online Reservation Systems

JSP and Servlets in Web Applications

Servlet

Introduction

A Servlet is a server-side Java program that processes client requests and generates dynamic responses. Servlets run inside a web server or servlet container such as Apache Tomcat. They are mainly used to handle HTTP requests and responses in web applications.

Role of Servlets in Web Applications

1. Accept requests from clients (web browsers).

2. Process business logic.
3. Interact with databases using JDBC.
4. Generate dynamic content.
5. Manage user sessions.
6. Send responses back to clients.

Features of Servlets

- Platform independent.
- Efficient and scalable.
- Supports session management.
- Provides better performance than CGI.
- Can communicate with databases and other resources.

Servlet Life Cycle

1. Loading and Instantiation
2. init() Method
3. service() Method
4. destroy() Method

Applications of Servlets

- Online Banking Systems
- E-Commerce Websites
- Student Management Systems
- Reservation Systems
- Enterprise Web Applications

JSP (Java Server Pages)

Introduction

JSP (Java Server Pages) is a server-side technology used to create dynamic web pages. JSP allows Java code to be embedded within HTML pages, making web page development easier and more efficient.

JSP pages are translated into Servlets by the web container before execution.

Role of JSP in Web Applications

1. Creates dynamic web pages.
2. Displays data received from Servlets.

3. Reduces the amount of Java code in web pages.
4. Separates presentation logic from business logic.
5. Improves maintainability of web applications.

Features of JSP

- Easy to develop dynamic pages.
- Supports Java programming.
- Automatically converted into Servlets.
- Supports reusable components.
- Integrates easily with databases and Servlets.

JSP Life Cycle

1. Translation
2. Compilation
3. Loading
4. Initialization
5. Request Processing
6. Destruction

Applications of JSP

- Dynamic Websites
- E-Commerce Applications
- Online Portals
- Banking Applications
- Educational Websites

Relationship Between JSP and Servlets

Subject	**Servlet**	**JSP**
Purpose	Handles request processing and business logic	Handles presentation and user interface
Coding Style	Java code based	HTML with embedded Java code
Execution	Directly executed by container	Converted into Servlet before execution
Development	More complex for UI design	Easier for UI development

Role	Controller	View
Performance	Slightly faster	Slightly slower due to translation process

Use of JSP and Servlets Together

In a web application:

1. User sends a request through a browser.
2. Servlet receives and processes the request.
3. Servlet interacts with the database if required.
4. Servlet forwards the result to JSP.
5. JSP displays the result to the user in a web page.

Thus, Servlets handle the processing logic while JSP handles the presentation layer, making web applications more organized and maintainable.

Role of JDBC, JSP, and Servlets in Web Applications

Web applications developed using Java EE rely heavily on JDBC, Servlets, and JSP. These technologies work together to process user requests, interact with databases, and generate dynamic web pages. JDBC handles database communication, Servlets process requests and business logic, and JSP is responsible for presenting information to users.

Role of JDBC (Java Database Connectivity)

JDBC is a standard Java API used to connect Java applications with relational databases such as MySQL, Oracle, PostgreSQL, and SQL Server. It acts as an intermediary between the Java application and the database, allowing data to be stored, retrieved, modified, and deleted.

Roles of JDBC

1. **Database Connectivity**
    - JDBC establishes a connection between a Java application and a database.
    - It enables communication with different database systems using JDBC drivers.
2. **Execution of SQL Statements**
    - JDBC allows Java programs to execute SQL commands such as:
        - SELECT
        - INSERT
        - UPDATE
        - DELETE

        - CREATE
        - DROP

3. **Data Retrieval**
    - Data stored in the database can be retrieved and processed within Java applications.
    - Results are stored in a ResultSet object.
4. **Data Manipulation**
    - JDBC allows insertion, updating, and deletion of records from database tables.
5. **Transaction Management**
    - Supports COMMIT and ROLLBACK operations.
    - Maintains data consistency and integrity.
6. **Security**
    - PreparedStatement helps prevent SQL Injection attacks.
    - Ensures safer database operations.
7. **Database Independence**
    - The same JDBC code can work with different databases by changing the JDBC driver.

Importance of JDBC in Web Applications

Without JDBC, web applications cannot store or retrieve user information. It is used in applications such as online banking systems, e-commerce websites, student management systems, and hospital management systems where data persistence is required.

Role of Servlets in Web Applications

A Servlet is a server-side Java component that runs inside a web container such as Apache Tomcat. It receives client requests, processes them, and sends responses back to the client.

Servlets act as the controller component in web applications and manage the flow of data between the user interface and the database.

Roles of Servlets

1. **Handling Client Requests**
    - Receives HTTP requests from web browsers.
    - Processes user input submitted through forms.
2. **Business Logic Processing**
    - Performs calculations, validations, and decision-making operations.

- Implements application-specific logic.

3. **Communication with Database**
    - Uses JDBC to interact with databases.
    - Retrieves and stores information based on user requests.
4. **Generating Dynamic Responses**
    - Produces content dynamically according to user requirements.
    - Sends HTML or other formats as responses.
5. **Session Management**
    - Maintains user information across multiple requests.
    - Supports cookies and HttpSession.
6. **Request Forwarding and Redirection**
    - Transfers control to JSP pages for displaying results.
    - Redirects users to different pages when necessary.
7. **Improving Performance**
    - Multiple requests can be handled efficiently using threads.
    - Faster than traditional CGI programs.

Importance of Servlets in Web Applications

Servlets serve as the backbone of Java web applications. They process user requests, coordinate with databases, and control the application workflow. Almost every enterprise web application uses Servlets for request handling.

Role of JSP (Java Server Pages) in Web Applications

JSP is a server-side technology used to create dynamic web pages. It allows developers to combine HTML with Java code to generate content dynamically.

JSP mainly focuses on the presentation layer of a web application and simplifies the creation of user interfaces.

Roles of JSP

1. **Creating Dynamic Web Pages**
    - Generates web pages dynamically based on user requests and database content.
2. **Displaying Data**
    - Presents data received from Servlets in a user-friendly format.
3. **User Interface Development**

- Makes web page design easier using HTML, CSS, and Java.

4. **Reducing Java Code**
    - Simplifies web development compared to writing HTML directly in Servlets.
5. **Separation of Presentation Logic**
    - Keeps user interface code separate from business logic.
6. **Improving Maintainability**
    - Changes in page design can be made without affecting business logic.
7. **Supporting Reusability**
    - Provides reusable components and tags.

Importance of JSP in Web Applications

JSP simplifies front-end development by allowing developers to focus on page design and presentation. It is commonly used in online portals, banking systems, e-commerce websites, and educational applications.

Working of JDBC, Servlets, and JSP Together

The three technologies work together to create a complete web application.

Step 1: User Request

A user enters information in a web page and submits a request through a browser.

Step 2: Request Processing by Servlet

The Servlet receives the request and processes the user input.

Step 3: Database Interaction through JDBC

The Servlet uses JDBC to connect to the database and execute SQL queries.

Step 4: Database Response

The database processes the query and returns the required data.

Step 5: Data Transfer to JSP

The Servlet sends the retrieved data to a JSP page.

Step 6: Dynamic Page Generation

JSP generates a dynamic web page using the received data.

Step 7: Response to User

The generated page is sent back to the browser and displayed to the user.

Diagram

Browser → Servlet → JDBC → Database

Browser ← JSP ← Servlet ← Database

Summary of Responsibilities

Technology	Main Responsibility
JDBC	Database connectivity and database operations
Servlet	Request processing and business logic
JSP	Presentation layer and dynamic web page generation

Together, JDBC, Servlets, and JSP form the foundation of Java-based web application development by providing data management, request handling, and user interface generation.

M2 – JDBC (Java Database Connectivity)

What is JDBC

JDBC (Java Database Connectivity)

JDBC (Java Database Connectivity) is a standard Java API that enables Java applications to communicate with relational databases. It provides methods and interfaces for connecting to a database, executing SQL statements, retrieving data, and updating records.

JDBC acts as a bridge between Java applications and database management systems (DBMS) such as MySQL, Oracle, PostgreSQL, SQL Server, and others.

Using JDBC, developers can perform various database operations directly from Java programs without depending on database-specific programming languages.

Need of JDBC

1. Database Connectivity

JDBC allows Java applications to establish a connection with different databases and exchange information efficiently.

2. Execution of SQL Queries

It enables execution of SQL commands such as SELECT, INSERT, UPDATE, DELETE, CREATE, and DROP.

3. Data Retrieval

JDBC helps retrieve data stored in databases and makes it available to Java applications for processing.

4. Data Manipulation

Users can add, modify, and delete records from database tables through Java programs.

5. Platform Independence

JDBC provides a common interface for database operations regardless of the underlying database system.

6. Transaction Management

It supports transaction control through COMMIT and ROLLBACK operations, ensuring data consistency.

7. Security

PreparedStatement in JDBC helps prevent SQL Injection attacks and improves application security.

8. Enterprise Application Development

Most web applications, banking systems, ERP software, and e-commerce platforms use JDBC for database interaction.

Advantages of JDBC

1. Easy database connectivity.
2. Supports multiple databases.
3. Platform independent.
4. Improves application security.
5. Supports transaction management.
6. Efficient data retrieval and manipulation.
7. Widely used in enterprise applications.

Different Types of JDBC Drivers

JDBC Driver

A JDBC Driver is a software component that enables Java applications to communicate with databases. It converts JDBC calls into database-specific commands that the database can understand.

There are four types of JDBC Drivers.

Type 1 Driver: JDBC-ODBC Bridge Driver

Introduction

The JDBC-ODBC Bridge Driver translates JDBC calls into ODBC calls and then forwards them to the database.

Working

Java Application → JDBC Driver → ODBC Driver → Database

Advantages

1. Easy to use.
2. Suitable for small applications.
3. No database-specific driver required.

Disadvantages

1. Performance is slow.
2. Requires ODBC installation.
3. Platform dependent.
4. Removed from modern Java versions.

Type 2 Driver: Native API Driver

Introduction

This driver converts JDBC calls into database-specific native API calls.

Working

Java Application → JDBC Driver → Native Database API → Database

Advantages

1. Better performance than Type 1.
2. Uses native database libraries.

Disadvantages

1. Requires installation of native libraries.
2. Platform dependent.
3. Difficult maintenance.

Type 3 Driver: Network Protocol Driver

Introduction

This driver sends JDBC calls to a middleware server, which converts them into database-specific protocols.

Working

Java Application → JDBC Driver → Middleware Server → Database

Advantages

1. Platform independent.

2. No native libraries required.
3. Supports multiple databases.

Disadvantages

1. Requires middleware server.
2. Increased network overhead.
3. Performance depends on middleware.

Type 4 Driver: Thin Driver

Introduction

Type 4 Driver directly converts JDBC calls into database-specific network protocols and communicates directly with the database server.

Working

Java Application → JDBC Driver → Database

Advantages

1. Fastest JDBC driver.
2. Platform independent.
3. No middleware required.
4. Easy deployment.
5. Most widely used driver.

Disadvantages

1. Database-specific driver required.

Examples

- MySQL Connector/J
- Oracle Thin Driver
- PostgreSQL JDBC Driver
- SQL Server JDBC Driver

Comparison of JDBC Drivers

Subject	**Type 1**	**Type 2**	**Type 3**	**Type 4**
Name	JDBC-ODBC Bridge	Native API Driver	Network Protocol Driver	Thin Driver

Speed	Slow	Moderate	Good	Fastest
Platform Dependency	Yes	Yes	No	No
Middleware Required	No	No	Yes	No
Native Library Required	No	Yes	No	No
Security	Low	Medium	High	High
Ease of Deployment	Difficult	Difficult	Moderate	Easy
Usage Today	Obsolete	Rare	Limited	Most Common

Diagram

Type 1

Java Application → JDBC Driver → ODBC Driver → Database

Type 2

Java Application → JDBC Driver → Native API → Database

Type 3

Java Application → JDBC Driver → Middleware Server → Database

Type 4

Java Application → JDBC Driver → Database

JDBC Architecture

Introduction

JDBC (Java Database Connectivity) Architecture is a framework that enables Java applications to communicate with databases. It provides a standard method for connecting to databases, executing SQL queries, and retrieving results.

Components of JDBC Architecture

1. Java Application

The client program that sends requests to the database and processes the returned data.

2. JDBC API

A set of classes and interfaces that provide methods for database connectivity and SQL execution.

3. DriverManager

Manages JDBC drivers and establishes connections between Java applications and databases.

4. JDBC Driver

Converts JDBC calls into database-specific commands and communicates with the database.

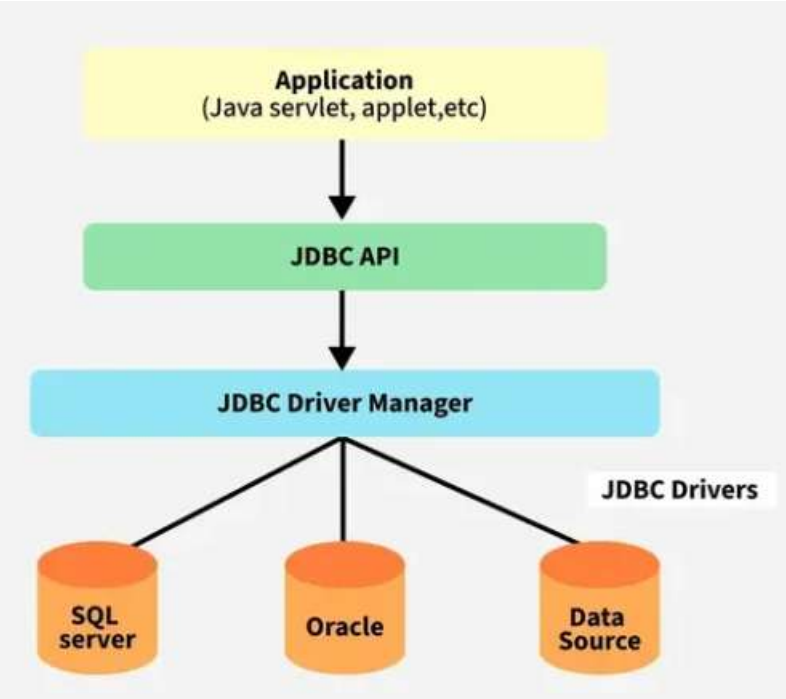
5. Database

Stores data and executes SQL queries requested by the application.

Working of JDBC Architecture

1. The Java application sends a request using JDBC API.
2. JDBC API forwards the request to DriverManager.
3. DriverManager loads the appropriate JDBC Driver.
4. The JDBC Driver communicates with the database.
5. The database executes the SQL query.
6. Results are returned through the JDBC Driver to the Java application.

Diagram



Advantages

1. Database-independent connectivity.
2. Easy execution of SQL queries.
3. Supports multiple databases.
4. Provides secure database access.
5. Suitable for web and enterprise applications.

Steps to Connect a Java Program to a Database

Introduction

To perform database operations in Java, a connection must be established between the Java application and the database. JDBC provides a standard mechanism for connecting Java programs to different databases and executing SQL statements.

Steps to Connect a Database

1. Import JDBC Package

Import the required classes from the java.sql package.

```
import java.sql.*;
```

2. Load the JDBC Driver

Load and register the database driver with the JVM.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

3. Establish Connection

Create a connection between the Java application and the database.

```
Connection con = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/studentdb",
"root",
"password");
```

4. Create Statement Object

Create a Statement object to execute SQL queries.

```
Statement stmt = con.createStatement();
```

5. Execute SQL Query

Execute the required SQL statement.

```
ResultSet rs = stmt.executeQuery("SELECT * FROM student");
```

6. Process the Result

Read and process the data returned by the database.

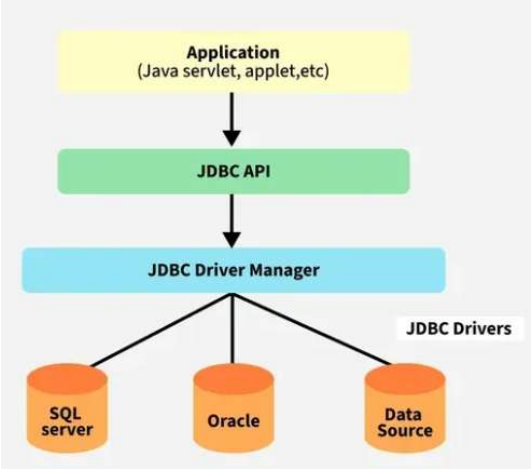
```
while(rs.next())
{
   System.out.println(rs.getString("name"));
}
```

7. Close Resources

Close ResultSet, Statement, and Connection objects after use.

con.close();

Diagram



Advantages

1. Enables communication between Java applications and databases.
2. Supports different database systems.
3. Provides secure and reliable data access.
4. Allows execution of SQL statements.
5. Supports enterprise-level applications.

Executing SELECT, INSERT, UPDATE and DELETE Queries in JDBC

SELECT Query

The SELECT statement is used to retrieve data from one or more tables in a database. It does not modify the database; it only fetches records that satisfy the specified condition. The result of a SELECT query is stored in a ResultSet object.

Method Used

executeQuery()

Example

```
String sql = "SELECT * FROM student";

ResultSet rs = stmt.executeQuery(sql);

while(rs.next())
{
  System.out.println(rs.getInt("id")+" "+
            rs.getString("name"));
}
```

INSERT Query

The INSERT statement is used to add new records into a database table. It increases the number of rows in the table by inserting new data.

Method Used

executeUpdate()

Example

```
String sql =
"INSERT INTO student VALUES(101,'Anik')";

int row = stmt.executeUpdate(sql);
```

UPDATE Query

The UPDATE statement is used to modify existing records in a database table. It changes the values of one or more columns based on a specified condition.

Method Used

executeUpdate()

Example

```
String sql =
"UPDATE student SET name='Rahul' WHERE id=101";

int row = stmt.executeUpdate(sql);
```

DELETE Query

The DELETE statement is used to remove one or more records from a database table. Records are deleted based on a specified condition.

Method Used

executeUpdate()

Example

```
String sql =
"DELETE FROM student WHERE id=101";

int row = stmt.executeUpdate(sql);
```

Difference Between Statement and PreparedStatement

Both Statement and PreparedStatement are interfaces of JDBC used to execute SQL queries. Statement is used for simple SQL queries, whereas PreparedStatement is used for precompiled and parameterized SQL queries.

Subject	Statement	PreparedStatement

Definition	Used to execute simple SQL queries.	Used to execute precompiled SQL queries with parameters.
Query Compilation	Query is compiled every time it is executed.	Query is compiled only once and reused multiple times.
Performance	Slower because query compilation occurs repeatedly.	Faster because query is precompiled.
Parameters	Does not support parameters.	Supports parameters using (?) placeholders.
Security	Vulnerable to SQL Injection attacks.	Protects against SQL Injection attacks.
Execution Speed	Slower for repeated execution.	Faster for repeated execution.
Reusability	Query must be written every time.	Same query can be reused with different values.
Dynamic Values	Values are directly concatenated into SQL statements.	Values are supplied separately using setter methods.
Efficiency	Less efficient.	More efficient.
Usage	Suitable for simple and static queries.	Suitable for dynamic and frequently executed queries.

Statement Example

```
Statement stmt = con.createStatement();

ResultSet rs =
stmt.executeQuery("SELECT * FROM student");
```

PreparedStatement Example

```
PreparedStatement ps =
con.prepareStatement(
"SELECT * FROM student WHERE id=?");

ps.setInt(1,101);

ResultSet rs = ps.executeQuery();
```

Advantages of PreparedStatement

1. Better performance.
2. Prevents SQL Injection.
3. Supports parameterized queries.
4. Easy to reuse.

5. More secure and efficient than Statement.

ResultSet and Metadata

ResultSet

Introduction

ResultSet is an interface in JDBC that stores the data returned by a SELECT query. It allows programmers to access and process database records row by row.

A ResultSet object is created when the executeQuery() method is used.

Example

ResultSet rs = stmt.executeQuery("SELECT * FROM student");

Types of ResultSet

1. Forward Only ResultSet

The cursor moves only in the forward direction.

ResultSet.TYPE_FORWARD_ONLY

Features:

- Default ResultSet type.
- Fast execution.
- Cannot move backward.

2. Scroll Insensitive ResultSet

The cursor can move forward and backward. Changes made in the database after ResultSet creation are not reflected.

ResultSet.TYPE_SCROLL_INSENSITIVE

Features:

- Supports random access.
- Database changes are not visible.

3. Scroll Sensitive ResultSet

The cursor can move in both directions and reflects database changes.

ResultSet.TYPE_SCROLL_SENSITIVE

Features:

- Supports forward and backward movement.
- Database changes are visible.

Common ResultSet Methods

Method	Purpose
next()	Moves to next row
previous()	Moves to previous row
first()	Moves to first row
last()	Moves to last row
getInt()	Retrieves integer value
getString()	Retrieves string value
close()	Closes ResultSet

Metadata

Introduction

Metadata means "data about data." It provides information about the structure and properties of a database, tables, columns, and query results.

JDBC provides two important metadata interfaces:

1. DatabaseMetaData
2. ResultSetMetaData

DatabaseMetaData

DatabaseMetaData provides information about the database, JDBC driver, supported features, and database structure.

It is obtained from the Connection object.

Example

DatabaseMetaData dbmd =

con.getMetaData();

Information Provided

- Database name
- Database version
- Driver name
- Driver version
- Available tables
- Supported SQL features

Common Methods

Method	Purpose
getDatabaseProductName()	Returns database name
getDatabaseProductVersion()	Returns database version
getDriverName()	Returns driver name
getDriverVersion()	Returns driver version
getTables()	Returns table information

ResultSetMetaData

ResultSetMetaData provides information about the columns present in a ResultSet object.

It is obtained from a ResultSet object.

Example

ResultSetMetaData rsmd =

rs.getMetaData();

Information Provided

- Number of columns
- Column names
- Data types
- Column size
- Table name

Common Methods

Method	Purpose
getColumnCount()	Returns number of columns
getColumnName()	Returns column name
getColumnType()	Returns column data type
getColumnLabel()	Returns column label
getTableName()	Returns table name

Difference Between DatabaseMetaData and ResultSetMetaData

Subject	DatabaseMetaData	ResultSetMetaData

Purpose	Information about database	Information about query result
Obtained From	Connection object	ResultSet object
Scope	Entire database	Specific query result
Information	Database, driver, tables	Columns and data types
Interface	DatabaseMetaData	ResultSetMetaData

Transaction Management in JDBC

Introduction

Transaction Management in JDBC is used to execute multiple SQL statements as a single unit of work. It ensures that either all operations are completed successfully or none of them are executed. This helps maintain data consistency and integrity.

Example

In a bank transaction, money deducted from one account and added to another account should be treated as a single transaction. If one operation fails, all changes should be cancelled.

Auto-Commit Mode

By default, JDBC runs in Auto-Commit mode, where each SQL statement is automatically saved.

con.setAutoCommit(true);

To manage transactions manually:

con.setAutoCommit(false);

Commit Operation

The commit() method permanently saves all changes made during a transaction.

con.commit();

Rollback Operation

The rollback() method cancels all changes made during a transaction if an error occurs.

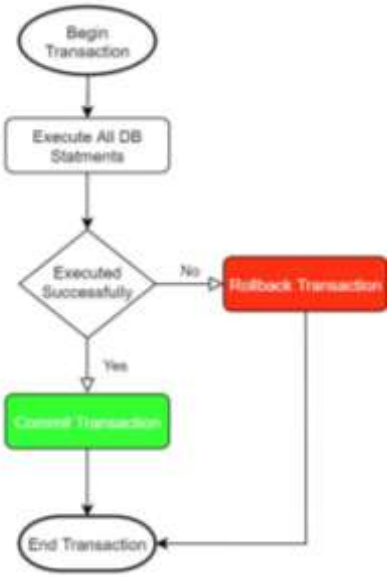
con.rollback();

Working of Transaction Management

1. Establish database connection.

2. Disable Auto-Commit mode.
3. Execute SQL statements.
4. Call commit() if all statements execute successfully.
5. Call rollback() if any statement fails.

Diagram



Advantages

1. Maintains data consistency.
2. Prevents partial updates.
3. Supports error recovery.
4. Ensures reliable database operations.
5. Widely used in banking and enterprise applications.

Commit, Rollback and Savepoint

Introduction

In JDBC, transaction management is used to maintain data consistency and integrity. Commit, Rollback, and Savepoint are important transaction control mechanisms that help manage database transactions effectively.

Commit

The commit() method is used to permanently save all changes made during a transaction to the database. Once a transaction is committed, the changes cannot be undone.

Syntax

con.commit();

Features

1. Permanently saves database changes.
2. Marks successful completion of a transaction.
3. Improves data consistency.
4. Used after successful execution of SQL statements.

Rollback

The rollback() method is used to undo all changes made during the current transaction. It restores the database to its previous state before the transaction began.

Syntax

con.rollback();

Features

1. Cancels all changes made in a transaction.
2. Used when an error occurs.
3. Prevents incomplete database updates.
4. Maintains data integrity.

Savepoint

A Savepoint is a checkpoint within a transaction. It allows a transaction to be rolled back to a specific point instead of rolling back the entire transaction.

Syntax

Savepoint sp = con.setSavepoint();

Features

1. Creates checkpoints within a transaction.
2. Allows partial rollback.
3. Reduces loss of completed operations.
4. Useful in large transactions.

Rollback to Savepoint

con.rollback(sp);

Difference Between Commit, Rollback and Savepoint

Subject	Commit	Rollback	Savepoint
Purpose	Saves changes permanently	Cancels changes	Creates a checkpoint
Effect	Makes changes permanent	Restores previous state	Allows partial rollback
Usage	After successful execution	When an error occurs	During long transactions
Data Recovery	Not possible after commit	Restores original data	Restores data up to savepoint

Example

Consider a bank transaction:

1. ₹1000 is deducted from Account A.
2. A Savepoint is created.
3. ₹1000 is added to Account B.
4. If an error occurs after the Savepoint, rollback can be done to that Savepoint.
5. If everything executes successfully, commit is performed.

Thus, Commit saves changes, Rollback cancels changes, and Savepoint provides a checkpoint for partial recovery during a transaction.

Batch Processing and SQL Injection in JDBC

Batch Processing in JDBC

Introduction

Batch Processing is a technique in JDBC that allows multiple SQL statements to be grouped together and executed as a single batch. Instead of sending each query separately to the database, all queries are sent together, which improves performance and reduces database communication overhead.

Advantages

1. Improves execution speed.
2. Reduces network traffic.
3. Minimizes database calls.
4. Efficient for bulk data operations.
5. Suitable for large-scale applications.

Methods Used

Method	Purpose
addBatch()	Adds SQL statement to batch
executeBatch()	Executes all statements in batch
clearBatch()	Removes statements from batch

Example

```
Statement stmt = con.createStatement();

stmt.addBatch(
"INSERT INTO student VALUES(101,'Anik')");

stmt.addBatch(
"INSERT INTO student VALUES(102,'Rahul')");

stmt.addBatch(
"INSERT INTO student VALUES(103,'Amit')");


stmt.executeBatch();
```

Working

1. Create Statement object.
2. Add multiple SQL statements using addBatch().
3. Execute all statements using executeBatch().
4. Database processes all queries together.

SQL Injection

Introduction

SQL Injection is a security attack in which malicious SQL code is inserted into user input to manipulate database queries. Attackers can access, modify, or delete sensitive data by injecting SQL commands.

Example of SQL Injection

Suppose a login query is written as:

```
String sql =

"SELECT * FROM user WHERE username='"

+ user +

"' AND password='" + pass + "'";
```

If the user enters:

' OR '1'='1

The query may become:

SELECT * FROM user

WHERE username='' OR '1'='1'

Since the condition is always true, unauthorized access may be granted.

Problems Caused by SQL Injection

1. Unauthorized access to data.
2. Data modification.
3. Data deletion.
4. Security breaches.
5. Loss of confidential information.

PreparedStatement and Prevention of SQL Injection

PreparedStatement is a JDBC interface used to execute parameterized SQL queries. It separates SQL code from user input, preventing malicious SQL commands from being executed.

Example

```
PreparedStatement ps =
con.prepareStatement(
"SELECT * FROM user WHERE username=? AND password=?");

ps.setString(1,user);
ps.setString(2,pass);

ResultSet rs = ps.executeQuery();
```

How PreparedStatement Prevents SQL Injection

1. SQL query is precompiled.
2. User input is treated as data, not SQL code.
3. Special characters are automatically handled.
4. Malicious SQL statements cannot alter the query structure.
5. Provides secure database access.

Advantages of PreparedStatement

1. Prevents SQL Injection attacks.
2. Improves performance.
3. Supports parameterized queries.

4. Enhances security.
5. Easy to reuse multiple times.

M3 – Java Server Pages (JSP)

JSP and JSP Life Cycle

JSP (Java Server Pages)

Introduction

JSP (Java Server Pages) is a server-side technology used to create dynamic web pages. It allows developers to embed Java code within HTML pages. JSP simplifies web development by separating presentation logic from business logic.

JSP pages are executed on the server and the generated output is sent to the client's web browser. Internally, every JSP page is converted into a Servlet by the web container before execution.

Advantages of JSP

1. Easy to create dynamic web pages.
2. Reduces the amount of Java code in web applications.
3. Separates presentation logic from business logic.
4. Supports reusable components and tags.
5. Easy integration with Servlets and databases.
6. Platform independent.
7. Faster development and maintenance.
8. Automatically converted into Servlets by the container.
9. Supports session tracking.
10. Suitable for enterprise web applications.

Applications

1. E-Commerce Websites
2. Banking Applications
3. Educational Portals
4. Online Reservation Systems
5. Enterprise Web Applications

JSP Life Cycle

Introduction

The JSP Life Cycle describes the stages through which a JSP page passes from creation to destruction. The JSP container manages the entire life cycle.

Phases of JSP Life Cycle

1. Translation Phase

The JSP page is translated into a Servlet source file by the JSP container.

example.jsp → example.java

2. Compilation Phase

The generated Servlet source file is compiled into bytecode.

example.java → example.class

3. Class Loading

The compiled Servlet class is loaded into memory by the container.

4. Instantiation

An object of the Servlet class is created.

5. Initialization

The jspInit() method is called only once when the JSP page is loaded.

```
public void jspInit()
{
  // Initialization code
}
```

6. Request Processing

The _jspService() method is called for every client request. It processes requests and generates responses.

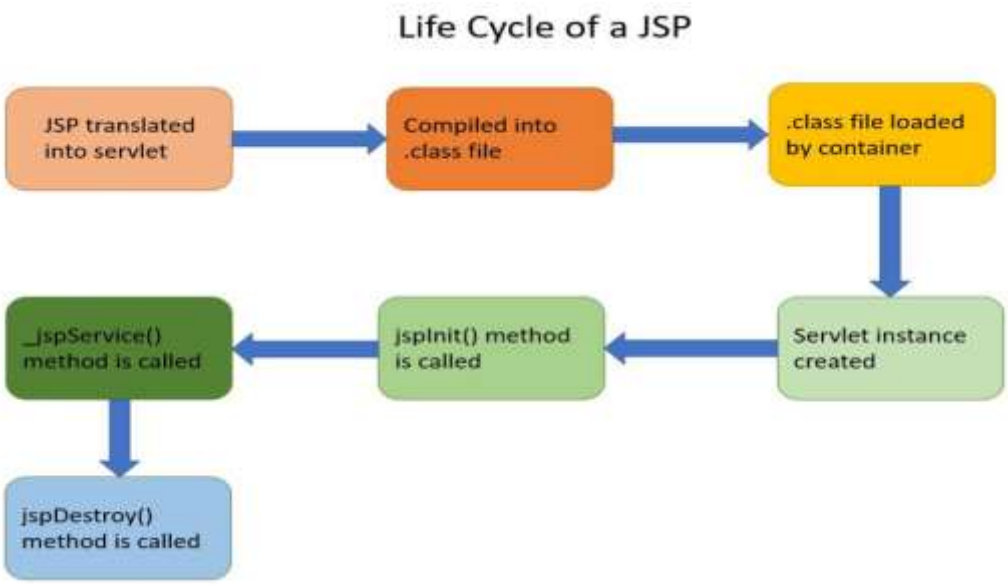
```
public void _jspService()
{
  // Request processing
}
```

7. Destruction

Before removing the JSP page from memory, the jspDestroy() method is called.

```
public void jspDestroy()
{
  // Cleanup code
}
```

JSP Life Cycle Diagram



JSP Scriptlet, Expression, Declaration and JSP Implicit Objects

JSP Scripting Elements

JSP provides scripting elements that allow Java code to be embedded within a JSP page. The three main scripting elements are Scriptlet, Expression, and Declaration.

JSP Scriptlet

Introduction

A Scriptlet is used to write Java code inside a JSP page. The code written inside a scriptlet is placed within the _jspService() method by the JSP container.

Syntax

<%

Java Code

%>

Example

<%

int a = 10;

int b = 20;

out.println(a+b);

%>

Uses

1. Variable declaration.

2. Conditional statements.
3. Loops and calculations.
4. Business logic implementation.

JSP Expression

Introduction

A JSP Expression is used to display the value of a variable or expression directly in the browser. The result is automatically converted into a string and sent to the output.

Syntax

<%= expression %>

Example

<%= new java.util.Date() %>

<%= 10 + 20 %>

Uses

1. Display variable values.
2. Display calculation results.
3. Show dynamic content.

JSP Declaration

Introduction

A JSP Declaration is used to declare variables and methods that become members of the generated Servlet class.

Syntax

<%! declaration %>

Example

```
<%!
int count = 0;

public int square(int n)
{
   return n*n;
}
%>
```

Uses

1. Declare instance variables.
2. Define methods.
3. Create reusable code within JSP.

Difference Between Scriptlet, Expression and Declaration

Subject	Scriptlet	Expression	Declaration
Tag	<% %>	<%= %>	<%! %>
Purpose	Write Java code	Display output	Declare variables and methods
Output	Not displayed automatically	Displayed automatically	Not displayed automatically
Placement	Inside _jspService()	Inside _jspService()	Outside _jspService()
Usage	Logic and processing	Dynamic output	Variable and method declaration

---

JSP Implicit Objects

Introduction

Implicit Objects are predefined objects created automatically by the JSP container. These objects can be used directly in JSP pages without explicit declaration.

JSP provides nine implicit objects:

- request
- response
- out
- session
- application
- config
- page
- pageContext
- exception

Five Important JSP Implicit Objects

1. request

The request object contains information sent by the client to the server.

Uses:

- Read form data.
- Read request parameters.
- Access client information.

Example:

String name = request.getParameter("name");

---

2. response

The response object is used to send data from the server to the client.

Uses:

- Redirect users.
- Send responses to browser.

Example:

response.sendRedirect("home.jsp");

---

3. out

The out object is used to display output on the browser.

Uses:

- Print text.
- Display dynamic content.

Example:

out.println("Welcome");

---

4. session

The session object stores user-specific information across multiple requests.

Uses:

- User login information.
- Shopping cart data.
- Session tracking.

Example:

session.setAttribute("user","Anik");

---

5. application

The application object represents the ServletContext and is shared by all users of the application.

Uses:

- Store application-wide data.
- Share information among users.

Example:

application.setAttribute("college","TIHC");

JSP Directives and JSP Action Elements

JSP Directives

Introduction

JSP Directives are instructions given to the JSP container that affect the overall structure and behavior of a JSP page. Directives are processed during the translation phase and do not produce any output directly.

Types of JSP Directives

1. Page Directive

The page directive provides information about the JSP page such as language, import statements, error pages, and content type.

Syntax

<%@ page attribute="value" %>

Example

<%@ page import="java.util.*" %>

---

2. Include Directive

The include directive inserts the content of another file into the current JSP page during translation time.

Syntax

<%@ include file="filename.jsp" %>

Example

<%@ include file="header.jsp" %>

Uses:

- Header files
- Footer files
- Common page content

---

3. Taglib Directive

The taglib directive is used to declare a custom tag library in a JSP page.

Syntax

<%@ taglib uri="URI" prefix="tagPrefix" %>

Example

<%@ taglib uri="http://java.sun.com/jsp/jstl/core"

prefix="c" %>

Uses:

- JSTL tags
- Custom tags
- Reusable components

JSP Action Elements

Introduction

JSP Action Elements are XML-based tags used to perform specific tasks during request processing. They execute at runtime and help in controlling the behavior of JSP pages.

Common JSP Action Elements

- jsp:include
- jsp:forward
- jsp:param
- jsp:useBean
- jsp:setProperty
- jsp:getProperty

---

1. jsp:include

The jsp:include action includes another resource into the current JSP page at request time.

Syntax

<jsp:include page="header.jsp" />

Example

<jsp:include page="menu.jsp" />

Uses:

- Dynamic inclusion of files.
- Reusable page components.

---

2. jsp:forward

The jsp:forward action forwards the request from one JSP page to another resource.

Syntax

<jsp:forward page="home.jsp" />

Example

<jsp:forward page="success.jsp" />

Uses:

- Page navigation.
- Request forwarding.

---

3. jsp:useBean

The jsp:useBean action creates or locates a JavaBean object for use in a JSP page.

Syntax

<jsp:useBean

id="student"

class="StudentBean" />

Example

<jsp:useBean

id="emp"

class="EmployeeBean" />

Uses:

- JavaBean integration.
- Reusable business objects.

Difference Between JSP Directives and JSP Action Elements

Subject	JSP Directives	JSP Action Elements
Processing Time	Translation Time	Request Time
Purpose	Provide instructions to JSP container	Perform actions during execution
Output Generation	Do not generate output directly	May generate output
Syntax	<%@ ... %>	jsp:...
Execution	Before JSP execution	During JSP execution

JavaBeans in JSP and JSP Expression Language (EL)

JavaBeans in JSP

Introduction

JavaBeans are reusable Java classes used to encapsulate data and business logic. A JavaBean contains private properties, public getter and setter methods, and a default constructor.

In JSP, JavaBeans help separate business logic from presentation logic, making web applications easier to develop and maintain.

Features of JavaBeans

1. Reusable component.
2. Encapsulates data.
3. Supports getter and setter methods.
4. Easy integration with JSP.
5. Improves code maintainability.

Conditions for a JavaBean

1. Class must implement Serializable (optional but recommended).
2. Must have a default constructor.
3. Properties should be private.
4. Public getter and setter methods must be provided.

Example of JavaBean

```
public class Student
{
  private String name;

  public Student(){}

  public String getName()
```

```
  {
    return name;
  }

  public void setName(String name)
  {
    this.name = name;
  }
}
```

Using JavaBean in JSP

Creating Bean

```
<jsp:useBean id="s"

class="Student" />
```

Setting Property

```
<jsp:setProperty

name="s"

property="name"

value="Anik" />
```

Getting Property

```
<jsp:getProperty

name="s"

property="name" />
```

Advantages

1. Promotes code reusability.
2. Separates business logic from presentation.
3. Simplifies JSP development.
4. Easy maintenance.
5. Reduces code duplication.

JSP Expression Language (EL)

Introduction

Expression Language (EL) is a simple scripting language used in JSP to access and display data stored in JavaBeans, request objects, session objects, and application objects.

EL reduces the need for Java code in JSP pages and makes web pages easier to read and maintain.

Syntax

${expression}

Examples

${student.name}

${10+20}

${sessionScope.user}

Features of JSP EL

1. Simple and easy syntax.
2. Reduces Java code in JSP pages.
3. Accesses JavaBean properties directly.
4. Supports arithmetic operations.
5. Supports logical and relational operations.
6. Accesses request, session, and application data.
7. Improves readability and maintainability.
8. Automatically converts data types when required.

EL Operators

Operator Type	**Examples**
Arithmetic	+, -, *, /, %
Relational	==, !=, >, <, >=, <=
Logical	&&, ||, !
Conditional	? :

Advantages

1. Easy to learn and use.
2. Reduces scripting code.
3. Improves page readability.
4. Supports dynamic content generation.
5. Integrates easily with JavaBeans and JSP components.

JSTL Core Tags

Introduction

JSTL (JavaServer Pages Standard Tag Library) is a collection of custom tags that simplifies JSP development. JSTL reduces the use of Java code in JSP pages and provides tags for common tasks such as displaying data, conditional processing, variable handling, and looping.

The JSTL Core Tag Library is declared using:

<%@ taglib uri="http://java.sun.com/jsp/jstl/core"

prefix="c" %>

<c:out>

The <c:out> tag is used to display data on a JSP page. It works similarly to out.println().

Syntax

<c:out value="${expression}" />

Example

<c:out value="${student.name}" />

Uses

1. Displays dynamic data.
2. Replaces JSP expressions.
3. Improves code readability.

<c:set>

The <c:set> tag is used to create and assign values to variables.

Syntax

<c:set var="variableName" value="value" />

Example

<c:set var="name" value="Anik" />

Uses

1. Creates variables.
2. Stores values for later use.
3. Supports EL expressions.

<c:if>

The <c:if> tag is used for conditional execution of code.

Syntax

<c:if test="${condition}">

  Statements

</c:if>

Example

```
<c:if test="${marks >= 40}">
  Pass
</c:if>
```

Uses

1. Decision making.
2. Conditional display of content.
3. Replaces Java if statements.

<c:choose>

The <c:choose> tag works like a switch statement in Java. It is used with <c:when> and <c:otherwise>.

Syntax

```
<c:choose>
  ...
</c:choose>
```

Uses

1. Multiple condition checking.
2. Alternative decision making.

<c:when>

The <c:when> tag specifies a condition inside <c:choose>.

Syntax

```
<c:when test="${condition}">
  Statements
</c:when>
```

Example

```
<c:when test="${marks >= 80}">
  Grade A
</c:when>
```

Uses

1. Checks specific conditions.
2. Similar to "case" in switch statement.

<c:otherwise>

The <c:otherwise> tag executes when none of the <c:when> conditions are true.

Syntax

```
<c:otherwise>
  Statements
</c:otherwise>
```

Example

```
<c:otherwise>
  Fail
</c:otherwise>
```

Uses

1. Default condition handling.
2. Similar to "default" in switch statement.

Example of c:choose, c:when and c:otherwise

```
<c:choose>

  <c:when test="${marks >= 80}">
    Grade A
  </c:when>

  <c:when test="${marks >= 60}">
    Grade B
  </c:when>

  <c:otherwise>
    Grade C
  </c:otherwise>

</c:choose>
```

<c:forEach>

The <c:forEach> tag is used to iterate over a collection or execute a loop repeatedly.

Syntax

```
<c:forEach var="i"
begin="1"
end="5">
  ...
</c:forEach>
```

Example

```
<c:forEach var="i" begin="1" end="5">
  ${i}
</c:forEach>
```

Uses

1. Loop execution.
2. Display collection data.
3. Replaces Java for loop.

Summary of JSTL Core Tags

Tag	Purpose
<c:out>	Displays output
<c:set>	Creates and assigns variables
<c:if>	Conditional execution
<c:choose>	Multiple condition checking
<c:when>	Defines a condition
<c:otherwise>	Default condition
<c:forEach>	Looping and iteration

JSTL Function Tags

JSTL Function Tags

Introduction

JSTL Function Tags provide a collection of predefined functions for performing string manipulation and other utility operations in JSP pages. These functions are available through the JSTL Functions Library.

Tag Library Declaration

<%@ taglib uri="http://java.sun.com/jsp/jstl/functions"

prefix="fn" %>

Common JSTL Function Tags

Function	Purpose
fn:length()	Returns length of a string or collection
fn:contains()	Checks whether a string contains another string
fn:startsWith()	Checks whether a string starts with specified text
fn:endsWith()	Checks whether a string ends with specified text
fn:toUpperCase()	Converts string to uppercase
fn:toLowerCase()	Converts string to lowercase
fn:trim()	Removes leading and trailing spaces
fn:replace()	Replaces characters or words
fn:substring()	Extracts a part of a string
fn:split()	Splits a string into an array

Examples

fn:length()

${fn:length("Advanced Java")}

fn:contains()

${fn:contains("Java Programming","Java")}

fn:toUpperCase()

${fn:toUpperCase("java")}

fn:replace()

${fn:replace("Java","J","K")}

Advantages

1. Reduces Java code in JSP.
2. Easy string manipulation.
3. Improves readability.
4. Works directly with Expression Language (EL).
5. Simplifies JSP development.

Custom Tag Library

Introduction

A Custom Tag Library is a collection of user-defined tags created by developers to perform specific tasks in JSP pages. Custom tags help in reusing code and reducing the amount of Java code written inside JSP pages.

Custom tags behave like built-in JSP tags but are developed according to application requirements.

Advantages

1. Promotes code reusability.
2. Reduces scripting code.
3. Improves maintainability.
4. Simplifies JSP development.
5. Separates business logic from presentation logic.

Steps to Create a Custom Tag

1. Create a Tag Handler Class.
2. Create a TLD (Tag Library Descriptor) file.
3. Register the tag library in JSP.
4. Use the custom tag in the JSP page.

Tag Handler Class Example

```
import javax.servlet.jsp.tagext.*;
import javax.servlet.jsp.*;

public class HelloTag extends TagSupport
{
  public int doStartTag()
  {
    try
    {
      pageContext.getOut()
            .print("Hello World");
    }
    catch(Exception e){}

    return SKIP_BODY;
  }
}
```

TLD File Example

```
<tag>
```

```
  <name>hello</name>
  <tag-class>HelloTag</tag-class>
  <body-content>empty</body-content>
</tag>
```

Using Custom Tag in JSP

```
<%@ taglib uri="/WEB-INF/custom.tld"
prefix="my" %>

<my:hello/>
```

Output

Hello World

Applications

1. Form validation.
2. User authentication.
3. Data formatting.
4. Reusable UI components.
5. Enterprise web applications.

M4 – Servlets

Servlet

Introduction

A Servlet is a server-side Java program that runs inside a web container such as Apache Tomcat. It is used to handle client requests, process them, and generate dynamic responses for web applications.

Servlets are mainly used to develop dynamic web applications and act as a bridge between web clients and databases.

Features of Servlets

1. Platform independent.
2. Server-side technology.
3. Supports dynamic web content.
4. Efficient request handling.
5. Supports session management.
6. Easy integration with databases.

Advantages of Servlets

1. Better performance than CGI because a single servlet can handle multiple requests using threads.
2. Platform independent as it is based on Java.
3. Supports session tracking.
4. Provides better security.
5. Easy database connectivity using JDBC.
6. Scalable and robust for enterprise applications.
7. Reusable and maintainable code.

Applications

1. Online Banking Systems
2. E-Commerce Websites
3. Student Management Systems
4. Online Reservation Systems
5. Enterprise Web Applications

Servlet Life Cycle

Introduction

The Servlet Life Cycle describes the stages through which a servlet passes from creation to destruction. The web container manages the entire life cycle of a servlet.

Phases of Servlet Life Cycle

1. Loading and Instantiation

The servlet class is loaded by the web container and an object of the servlet is created.

2. Initialization

The init() method is called only once when the servlet is loaded.

```
public void init()
{
  // Initialization code
}
```

3. Request Processing

The service() method is called whenever a client sends a request. It processes the request and generates the response.

```
public void service(
```

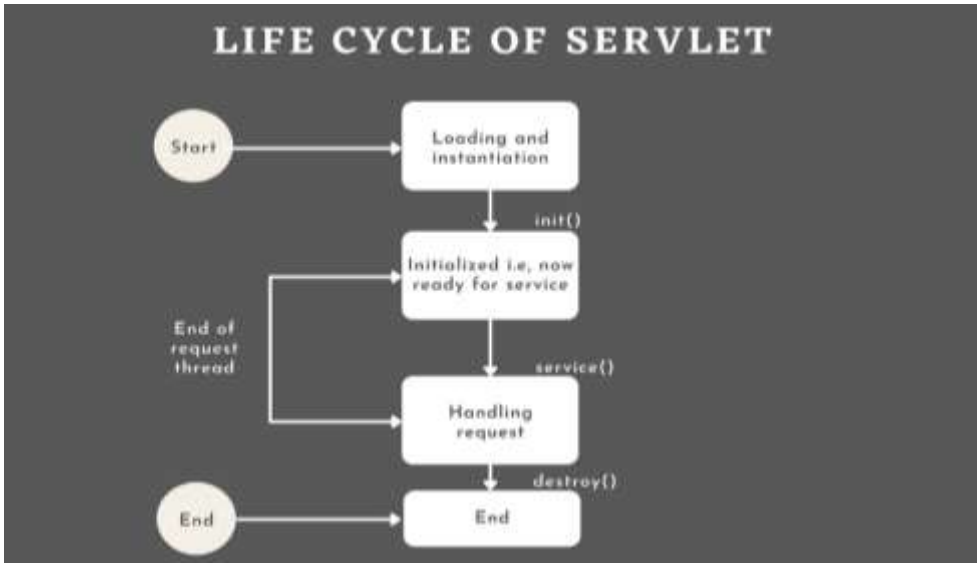
```
ServletRequest req,
ServletResponse res)
{
  // Request processing
}
```

4. Destruction

The destroy() method is called before removing the servlet from memory.

```
public void destroy()
{
  // Cleanup code
}
```

Servlet Life Cycle Diagram



GET and POST Request Handling in Servlets

Introduction

Servlets handle client requests using HTTP methods. The two most commonly used methods are GET and POST. These requests are processed using the doGet() and doPost() methods of the HttpServlet class.

GET Request

A GET request is used to retrieve information from the server. Data is sent as part of the URL.

Syntax

```
protected void doGet(
```

```
HttpServletRequest request,
HttpServletResponse response)
{
  // Processing code
}
```

Features

1. Used to retrieve data.
2. Data is visible in the URL.
3. Limited amount of data can be sent.
4. Faster than POST.
5. Less secure for sensitive information.

Example URL

http://localhost:8080/app?name=Anik

POST Request

A POST request is used to send data to the server for processing.

Syntax

```
protected void doPost(
HttpServletRequest request,
HttpServletResponse response)
{
  // Processing code
}
```

Features

1. Used to submit data.
2. Data is sent in the request body.
3. Large amount of data can be sent.
4. More secure than GET.
5. Suitable for login and registration forms.

Difference Between GET and POST

Subject	GET	POST
Purpose	Retrieve data	Send data
Data Location	URL	Request Body
Security	Less Secure	More Secure
Data Size	Limited	Large
Speed	Faster	Slightly Slower
Bookmarking	Possible	Not Possible
Usage	Search, View Data	Registration, Login, Forms

Servlet Deployment Descriptor (web.xml)

Introduction

The Deployment Descriptor is an XML file named **web.xml** used to configure web applications. It contains information about servlets, servlet mappings, initialization parameters, welcome files, and security settings.

The file is stored inside the **WEB-INF** directory of a web application.

Location

WebContent

```
 |
 └── WEB-INF
     |
     └── web.xml
```

Purpose of web.xml

1. Defines servlet configuration.
2. Maps URLs to servlets.
3. Specifies initialization parameters.
4. Configures welcome pages.
5. Defines security settings.
6. Manages session configuration.
7. Centralized configuration of web applications.

Example

<web-app>

```
<servlet>
  <servlet-name>Hello</servlet-name>
  <servlet-class>
    com.demo.HelloServlet
  </servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>Hello</servlet-name>
  <url-pattern>/hello</url-pattern>
</servlet-mapping>

</web-app>
```

Working

1. The web container reads the web.xml file.
2. Servlet information is loaded.
3. URL patterns are mapped to servlets.
4. Requests are forwarded to the appropriate servlet.

Advantages

1. Easy servlet configuration.
2. Centralized management.
3. Flexible URL mapping.
4. Supports security and session settings.
5. Simplifies web application deployment.

ServletContext

Introduction

ServletContext is an interface that provides information about the entire web application. It is created by the web container when the application starts and is shared among all servlets in the application.

A single ServletContext object is available for the entire web application.

Obtaining ServletContext

ServletContext context =

getServletContext();

Uses of ServletContext

1. Sharing data among multiple servlets.
2. Reading application-wide initialization parameters.
3. Accessing web application resources.
4. Obtaining server information.
5. Logging messages.
6. Managing application-level attributes.

Example

ServletContext context =

getServletContext();

context.setAttribute("college","TIHC");

Advantages

1. Shared by all servlets.
2. Stores application-wide data.
3. Reduces code duplication.
4. Easy resource management.
5. Improves communication among servlets.

ServletConfig

Introduction

ServletConfig is an interface used to provide configuration information to a specific servlet. Each servlet has its own ServletConfig object created by the web container.

It is used to read initialization parameters defined in the deployment descriptor.

Obtaining ServletConfig

ServletConfig config =

getServletConfig();

Example

String user =

config.getInitParameter("username");

Uses of ServletConfig

1. Reading servlet initialization parameters.
2. Providing servlet-specific configuration.
3. Accessing ServletContext object.
4. Configuring servlet behavior during initialization.

Advantages

1. Servlet-specific configuration.
2. Easy parameter management.
3. Improves flexibility.
4. Supports initialization settings.

Difference Between ServletConfig and ServletContext

Subject	ServletConfig	ServletContext
Definition	Provides configuration for a specific servlet	Provides information about the entire web application
Scope	Servlet Level	Application Level
Object Count	One object per servlet	One object per application
Data Sharing	Cannot be shared among servlets	Shared among all servlets
Parameters	Servlet initialization parameters	Application initialization parameters
Availability	Available to a particular servlet	Available to all servlets
Creation	Created for each servlet	Created once per application
Usage	Servlet-specific configuration	Application-wide communication and resource sharing

Session Management Techniques in Servlets

Introduction

Session Management is the process of maintaining user information across multiple requests. Since HTTP is a stateless protocol, servlets use session management techniques to track user activities during a session.

Session Management Techniques

1. Cookies

Cookies are small text files stored on the client's browser. They store user-specific information and are sent with each request.

2. URL Rewriting

In URL Rewriting, session information is appended to the URL.

Example

http://localhost:8080/app?userid=101

3. Hidden Form Fields

Hidden fields store session information inside HTML forms.

Example

<input type="hidden" name="userid" value="101">

4. HttpSession

HttpSession is the most commonly used session management technique. It stores user data on the server side.

Example

HttpSession session =

request.getSession();

session.setAttribute("user","Anik");

Advantages of Session Management

1. Maintains user state.
2. Tracks user activities.
3. Supports login and authentication.
4. Provides personalized user experience.
5. Essential for web applications such as banking and e-commerce systems.

Cookies

Introduction

Cookies are small text files stored in the client's browser. They are used to store user-specific information and help maintain session information between multiple requests.

Cookies are sent by the server to the browser and are automatically returned by the browser with subsequent requests.

Creating a Cookie

A cookie is created using the Cookie class.

Example

```
Cookie ck =
new Cookie("user","Anik");


response.addCookie(ck);
```

Reading a Cookie

Cookies can be read using the getCookies() method.

Example

```
Cookie cookies[] =
request.getCookies();


for(Cookie c : cookies)
{
  System.out.println(
    c.getName()+" = "+
    c.getValue());
}
```

Common Cookie Methods

Method	**Purpose**
getName()	Returns cookie name
getValue()	Returns cookie value
setValue()	Sets cookie value
setMaxAge()	Sets cookie lifetime
getMaxAge()	Returns cookie lifetime

Advantages of Cookies

1. Simple session tracking.
2. Stores user preferences.
3. Reduces server load.
4. Easy to implement.

5. Supports personalization of web applications.

Limitations of Cookies

1. Limited storage capacity.
2. Less secure for sensitive data.
3. Users can disable cookies.
4. Stored on the client side.

Servlet Chaining

Introduction

Servlet Chaining is a technique in which the output or request from one servlet is passed to another servlet for further processing. Multiple servlets work together sequentially to handle a client request.

Servlet chaining is commonly achieved using **RequestDispatcher** through forwarding or including resources.

Working

1. Client sends a request to the first servlet.
2. The first servlet processes the request.
3. Request is forwarded to another servlet.
4. The second servlet performs additional processing.
5. Response is sent to the client.

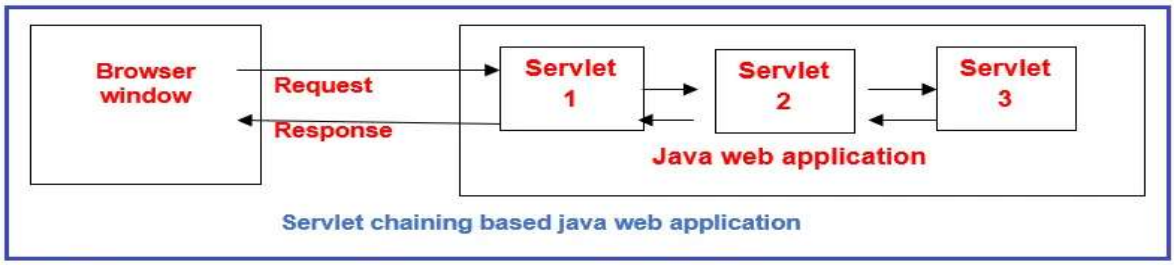
Example

```
RequestDispatcher rd =

request.getRequestDispatcher("SecondServlet");


rd.forward(request,response);
```

Advantages

1. Code reusability.
2. Better modularity.
3. Simplifies application design.
4. Multiple servlets can share tasks.
5. Easy maintenance.

Servlet Chaining Diagram



Servlet Filters

Introduction

A Servlet Filter is a Java component that intercepts client requests and server responses before they reach a servlet or after they leave a servlet.

Filters are used to perform preprocessing and postprocessing tasks without modifying the servlet code.

Purpose of Servlet Filters

1. Authentication and authorization.
2. Logging user activities.
3. Data compression.
4. Request validation.
5. Response modification.
6. Performance monitoring.
7. Security checking.

Filter Life Cycle Methods

init()

Called once when the filter is loaded.

```
public void init(FilterConfig fc)
{
}
```

doFilter()

Processes requests and responses.

```
public void doFilter(
ServletRequest req,
ServletResponse res,
FilterChain chain)
```

```
{
  chain.doFilter(req,res);
}
```

destroy()

Called before the filter is removed.

```
public void destroy()
{
}
```

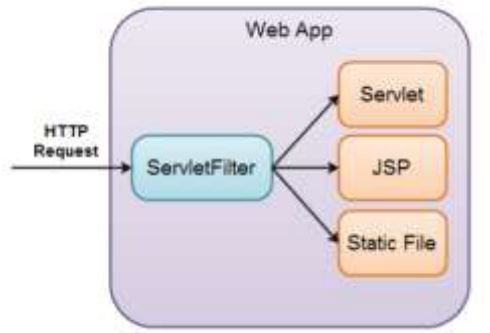
Working of Filter

1. Client sends request.
2. Filter intercepts the request.
3. Filter performs processing.
4. Request reaches servlet.
5. Servlet generates response.
6. Filter processes response.
7. Response sent to client.

Advantages

1. Improves security.
2. Promotes code reusability.
3. Reduces code duplication.
4. Centralized request processing.
5. Easy maintenance.

Servlet Filter Diagram



File Upload and Download Using Servlets

Introduction

Servlets provide support for uploading files from a client to a server and downloading files from a server to a client. This feature is commonly used in web applications such as document management systems, online forms, and file-sharing platforms.

Steps for File Upload

1. Create an HTML form with file input control.
2. Set form method as POST.
3. Set enctype as multipart/form-data.
4. Use @MultipartConfig annotation in the servlet.
5. Retrieve the uploaded file using the Part interface.
6. Save the file on the server.

Example

@MultipartConfig

Part filePart =

request.getPart("file");

Steps for File Download

1. Read the file from the server.
2. Set response content type.
3. Set response headers.
4. Write file data to OutputStream.
5. Send file to the client browser.

Example

response.setContentType(

"application/octet-stream");

Applications

1. Resume upload systems.
2. Online document sharing.
3. Student assignment submission.
4. Image upload applications.
5. File management systems.

Request Dispatching in Servlets

Introduction

Request Dispatching is the process of transferring a client request from one servlet to another servlet, JSP page, or web resource. It is performed using the RequestDispatcher interface.

Methods of Request Dispatching

1. forward()

The forward() method transfers the request completely to another resource. The control does not return to the original servlet.

Syntax

```
RequestDispatcher rd =

request.getRequestDispatcher(

"page.jsp");


rd.forward(

request,response);
```

Features

1. Transfers control completely.
2. URL remains unchanged.
3. Request object is shared.

---

2. include()

The include() method includes the output of another resource into the current response.

Syntax

```
RequestDispatcher rd =

request.getRequestDispatcher(

"page.jsp");


rd.include(

request,response);
```

Features

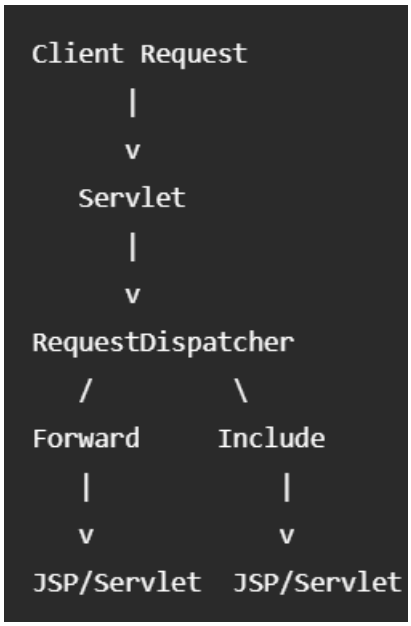
1. Includes another resource's output.

2. Control returns to the calling servlet.
3. Response contains output from both resources.

Difference Between forward() and include()

Subject	**forward()**	**include()**
Control	Transferred completely	Returns to caller
Response	Generated by target resource	Generated by both resources
Execution	Original servlet stops	Original servlet continues
Usage	Navigation	Content inclusion

Request Dispatching Diagram



M5 – JSP and Servlet Integration

Difference Between JSP and Servlet

Introduction

JSP (Java Server Pages) and Servlets are server-side technologies used for developing dynamic web applications. Both execute on the server and generate dynamic content, but they differ in their purpose and implementation.

Subject	**JSP**	**Servlet**
Full Form	Java Server Pages	Java Servlet
Purpose	Used for presentation layer	Used for business logic and request processing

Subject	JSP	Servlet
Coding Style	HTML with embedded Java code	Pure Java code
Development	Easier to develop web pages	More coding required
Execution	Internally converted into a Servlet	Executes directly as a Servlet
Maintenance	Easy to maintain UI pages	Difficult for UI design
Performance	Slightly slower initially	Faster execution
Suitable For	User Interface (View)	Controller and Processing Logic
File Extension	.jsp	.java
MVC Role	View	Controller

Advantages of JSP

1. Easy user interface design.
2. Less Java code.
3. Better readability.
4. Faster web page development.

Advantages of Servlet

1. Better performance.
2. Efficient request handling.
3. Easy integration with databases.
4. Suitable for business logic.

JSP and Servlets Working Together

Introduction

In a web application, JSP and Servlets work together following the MVC (Model-View-Controller) architecture. The Servlet acts as the Controller, while JSP acts as the View.

The Servlet receives requests, processes business logic, interacts with the database, and forwards results to a JSP page. The JSP page displays the results to the user.

Working Process

1. Client sends request.
2. Servlet receives the request.

3. Servlet processes business logic.
4. Servlet interacts with the database.
5. Data is stored in request/session objects.
6. Servlet forwards request to JSP.
7. JSP displays the result to the user.

Example

```
request.setAttribute(
"name","Anik");

RequestDispatcher rd =
request.getRequestDispatcher(
"welcome.jsp");

rd.forward(
request,response);
```

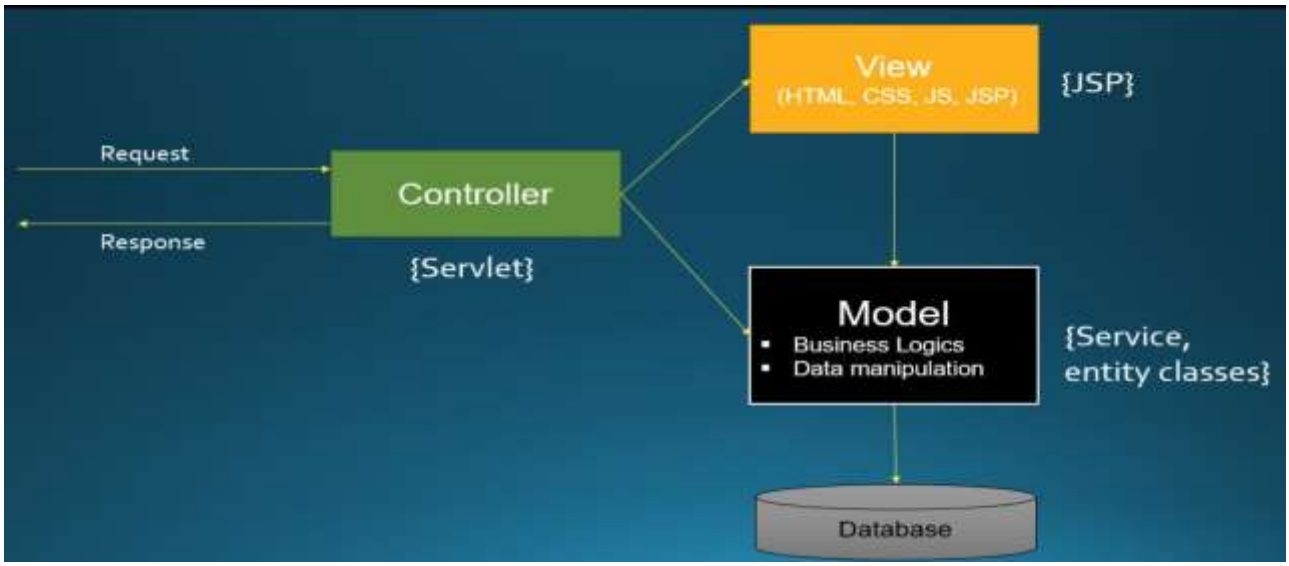
JSP Page

```
Welcome ${name}
```

Advantages

1. Separation of presentation and business logic.
2. Better maintainability.
3. Improved code reusability.
4. Easier application development.
5. Supports MVC architecture.

JSP-Servlet Interaction Diagram



Model–View–Controller (MVC) Architecture

Introduction

MVC (Model–View–Controller) is a software design pattern used to develop web applications. It separates an application into three components: Model, View, and Controller. This separation improves maintainability, reusability, and scalability.

Components of MVC

Model

The Model represents application data and business logic. It interacts with the database to store and retrieve information.

View

The View is responsible for displaying data to the user. In Java web applications, JSP pages usually act as the View.

Controller

The Controller handles user requests, processes input, interacts with the Model, and forwards results to the View. Servlets usually act as Controllers.

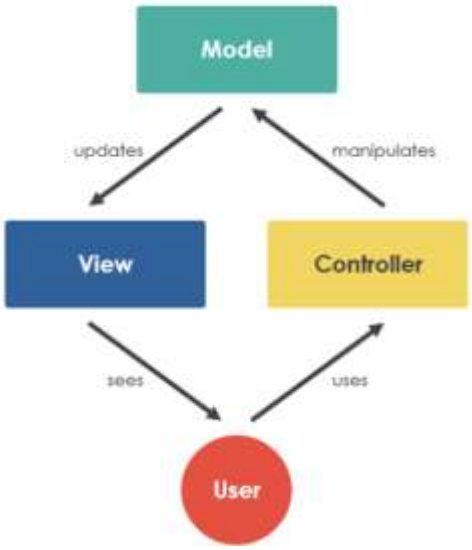
Working of MVC

1. User sends a request.
2. Controller receives the request.
3. Controller interacts with the Model.
4. Model accesses the database.
5. Data is returned to the Controller.
6. Controller forwards data to the View.
7. View displays the result to the user.

Advantages

1. Separation of concerns.
2. Easy maintenance.
3. Better code reusability.
4. Improved scalability.
5. Simplified development.

MVC Diagram



Request Forwarding Between JSP and Servlet

Introduction

Request Forwarding is a mechanism that allows a Servlet or JSP to transfer a client request to another Servlet, JSP, or resource within the same application. It is performed using the RequestDispatcher interface.

The request object is shared between the source and destination resources.

Syntax

```
RequestDispatcher rd =
request.getRequestDispatcher(
"result.jsp");

rd.forward(
request,response);
```

Working

1. Client sends a request to a Servlet.
2. Servlet processes the request.
3. Data is stored in request attributes.
4. Servlet forwards the request to a JSP page.
5. JSP accesses the data and displays the result.

Example

Servlet

```
request.setAttribute(
"name","Anik");

RequestDispatcher rd =
```

```
request.getRequestDispatcher(
"welcome.jsp");

rd.forward(
request,response);
```

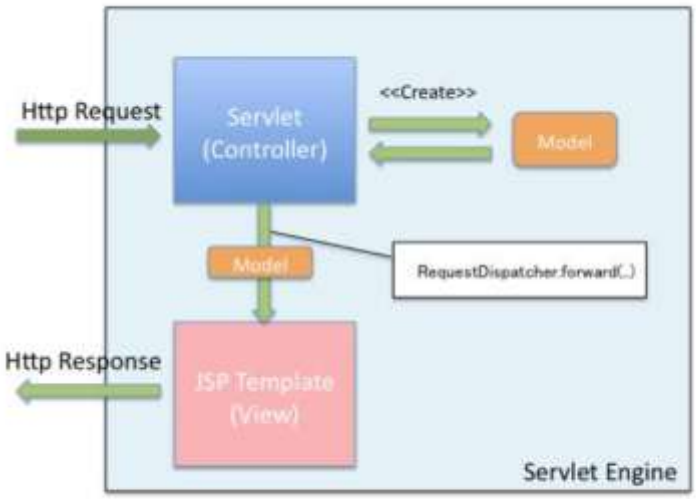
JSP

```
Welcome ${name}
```

Advantages

1. Easy communication between resources.
2. Data can be shared using request attributes.
3. Supports MVC architecture.
4. Reduces code duplication.
5. Improves application organization.

Request Forwarding Diagram



Role of MVC in Web Application Design

Introduction

MVC (Model–View–Controller) is a software design pattern used to develop web applications. It separates the application into three independent components: Model, View, and Controller. This separation makes the application easier to develop, maintain, and manage.

Components of MVC

Model

The Model handles application data and business logic. It interacts with the database to store and retrieve information.

View

The View is responsible for displaying data to the user. In Java web applications, JSP pages generally act as the View.

Controller

The Controller receives user requests, processes them, communicates with the Model, and forwards results to the View. Servlets generally act as Controllers.

Role of MVC in Web Application Design

1. Separation of Concerns

MVC separates business logic, user interface, and request handling into different components.

2. Easy Maintenance

Changes in one component do not affect the others, making maintenance easier.

3. Code Reusability

The same Model and View components can be reused in different parts of the application.

4. Better Organization

Application code remains well-structured and easy to understand.

5. Scalability

MVC supports the development of large and complex web applications.

6. Parallel Development

Different developers can work simultaneously on Model, View, and Controller components.

7. Improved Testing

Each component can be tested independently.

Working of MVC

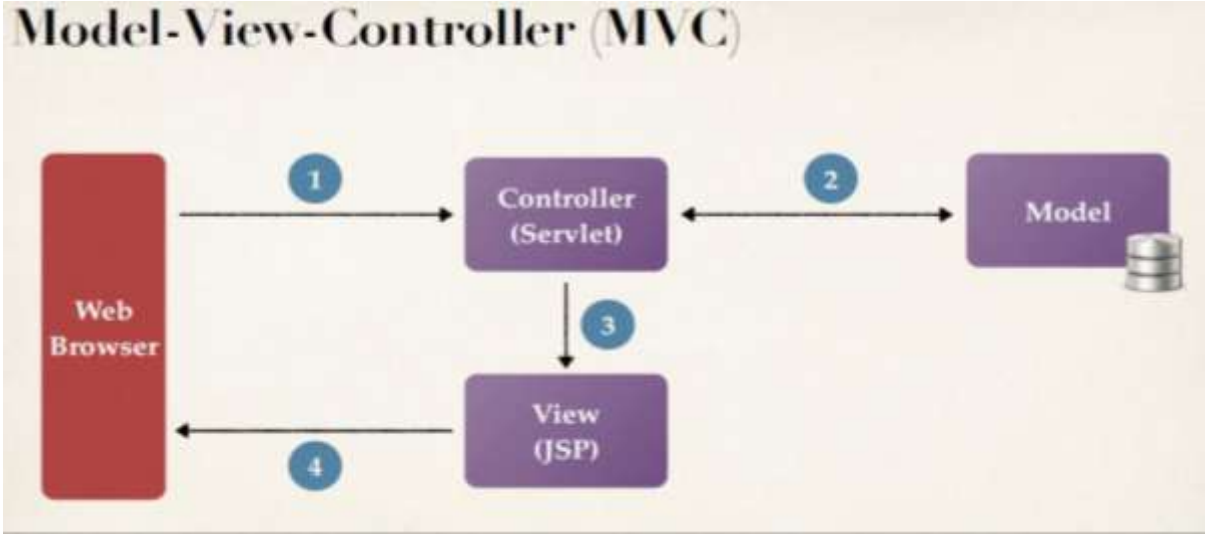
1. User sends a request.
2. Controller receives the request.
3. Controller interacts with the Model.
4. Model processes data and accesses the database.
5. Controller forwards data to the View.
6. View displays the result to the user.

Advantages

1. Better code management.
2. Improved maintainability.
3. High reusability.

4. Supports large applications.
5. Simplifies web application development.

MVC Diagram



M6 – Hibernate Framework

Hibernate

Introduction

Hibernate is an open-source ORM (Object Relational Mapping) framework for Java. It simplifies database operations by mapping Java objects to database tables. Hibernate eliminates most JDBC coding and provides a powerful mechanism for data persistence.

Using Hibernate, developers can perform database operations without writing complex SQL queries.

Features of Hibernate

1. Object Relational Mapping (ORM).
2. Database independence.
3. Automatic table mapping.
4. Supports HQL (Hibernate Query Language).
5. Automatic generation of SQL queries.
6. Caching mechanism for improved performance.
7. Transaction management support.
8. Reduces JDBC code.
9. Supports annotations and XML configuration.
10. Easy integration with Java EE applications.

Architecture Components

1. Configuration
2. SessionFactory
3. Session
4. Transaction
5. Query
6. Persistent Objects

Advantages of Hibernate

1. Simplifies database programming.
2. Reduces boilerplate JDBC code.
3. Improves application maintainability.
4. Provides better performance through caching.
5. Supports object-oriented programming concepts.
6. Easy database portability.
7. Faster development process.

Advantages of ORM over JDBC

Introduction

ORM (Object Relational Mapping) is a technique that maps Java objects to database tables. Hibernate is a popular ORM framework. Compared to JDBC, ORM provides a higher-level approach for database interaction.

Advantages of ORM over JDBC

1. Less Coding

ORM automatically generates SQL statements, whereas JDBC requires manual SQL writing.

2. Object-Oriented Approach

ORM works directly with Java objects, while JDBC works with tables and records.

3. Database Independence

ORM applications can switch databases with minimal changes. JDBC often requires database-specific modifications.

4. Automatic Mapping

ORM automatically maps classes to database tables.

5. Better Maintainability

Code is easier to understand and maintain due to reduced SQL statements.

6. Built-in Caching

ORM frameworks provide caching mechanisms to improve performance.

7. Transaction Management

ORM offers easier transaction handling than JDBC.

8. Improved Productivity

Developers focus on business logic instead of database operations.

Difference Between ORM and JDBC

Subject	**ORM (Hibernate)**	**JDBC**
Programming Style	Object-Oriented	SQL-Oriented
SQL Writing	Mostly Automatic	Manual
Code Size	Less	More
Mapping	Automatic	Manual
Database Independence	High	Low
Caching	Available	Not Available
Development Speed	Faster	Slower
Maintenance	Easy	Difficult

Applications of ORM

1. Enterprise Applications
2. Banking Systems
3. E-Commerce Applications
4. Student Management Systems
5. Large-Scale Web Applications

Hibernate Architecture and Core Components

Introduction

Hibernate Architecture defines how Hibernate communicates between a Java application and the database. It provides a layer between the application and database, simplifying data persistence and retrieval.

Core Components of Hibernate

1. Configuration

Configuration is used to configure Hibernate settings and database connection information. It reads the hibernate.cfg.xml file.

2. SessionFactory

SessionFactory is a factory class responsible for creating Session objects. Generally, only one SessionFactory is created per application.

3. Session

Session is the primary interface used to perform database operations such as insert, update, delete, and retrieve records.

4. Transaction

Transaction ensures that database operations are executed as a single unit of work.

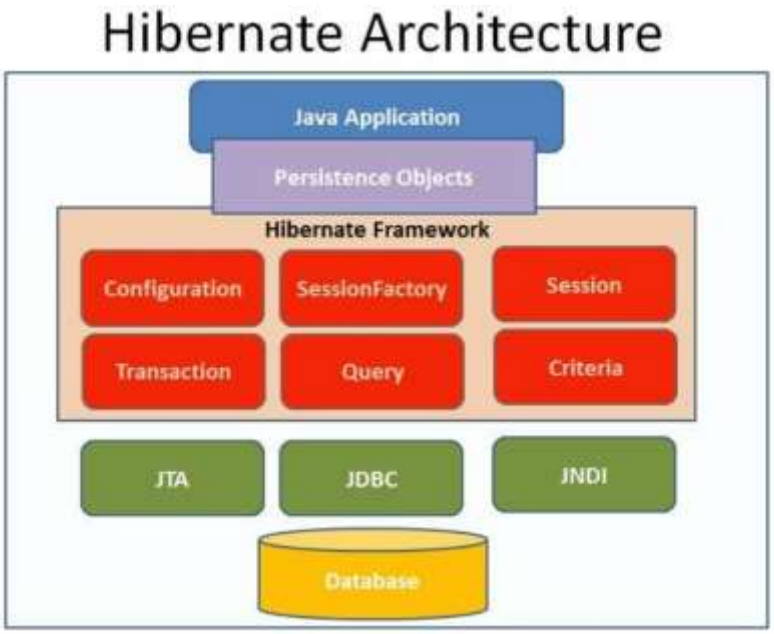
5. Query

Query is used to execute HQL (Hibernate Query Language) statements.

6. Persistent Objects

Persistent Objects are Java objects whose state is stored in the database.

Hibernate Architecture Diagram



Advantages

1. Simplifies database access.
2. Reduces JDBC coding.
3. Supports transaction management.
4. Provides caching support.
5. Improves application performance.

Steps to Set Up Hibernate in a Java Application

Introduction

To use Hibernate in a Java application, necessary libraries, configuration files, and mapping information must be provided.

Steps

1. Add Hibernate Libraries

Add Hibernate JAR files and database driver JAR files to the project.

2. Create Java Entity Class

Create a POJO class representing a database table.

Example

```
public class Student
{
  private int id;
  private String name;
}
```

3. Create Mapping Information

Map the Java class to a database table using annotations or XML.

Example

```
@Entity
@Table(name="student")
```

4. Create Hibernate Configuration File

Create hibernate.cfg.xml containing database connection details.

Example

```
<property name="connection.url">
jdbc:mysql://localhost/test
</property>
```

5. Create SessionFactory

Load configuration and create SessionFactory object.

```
Configuration cfg =
new Configuration();
```

```
SessionFactory sf =
cfg.configure()
  .buildSessionFactory();
```

6. Open Session

Create a Session object for database operations.

```
Session session =
sf.openSession();
```

7. Begin Transaction

```
Transaction tx =
session.beginTransaction();
```

8. Perform Database Operations

```
session.save(student);
```

9. Commit Transaction

```
tx.commit();
```

10. Close Session

```
session.close();
```

Summary of Setup Process

```
Add Hibernate Libraries
        |
        v
Create Entity Class
        |
        v
Configure Hibernate
        |
        v
Create SessionFactory
        |
        v
Open Session
        |
        v
Begin Transaction
        |
        v
Database Operation
        |
        v
```

Commit Transaction
|
v
Close Session

Mapping in Hibernate

Introduction
Mapping is the process of establishing a relationship between Java classes and database tables. Hibernate uses mapping to convert Java objects into database records and vice versa. Through mapping, each class corresponds to a table, class attributes correspond to columns, and object instances correspond to rows in the table.
Mapping Java Classes to Database Tables
Hibernate provides two ways of mapping:
1. Annotation-Based Mapping
Annotations are written directly in the Java class.
Example

```
@Entity
@Table(name="student")

public class Student
{
  @Id
  private int id;

  private String name;
}
```

2. XML-Based Mapping
Mapping information is stored in a separate XML file.
Example

```
<class name="Student"
table="student">

<id name="id"/>

<property name="name"/>

</class>
```

Benefits of Mapping

1. Reduces manual SQL coding.
2. Supports object-oriented programming.
3. Simplifies database operations.
4. Improves maintainability.
5. Enables automatic table mapping.

Hibernate Configuration Using XML

Introduction

Hibernate configuration is done using the **hibernate.cfg.xml** file. This file contains database connection information and Hibernate settings required to establish communication with the database.

The configuration file is usually placed inside the project's classpath.

Purpose of hibernate.cfg.xml

1. Database connection setup.
2. Hibernate property configuration.
3. Mapping file registration.
4. SessionFactory creation.
5. Transaction configuration.

Basic Structure of hibernate.cfg.xml

```
<?xml version="1.0"?>

<!DOCTYPE hibernate-configuration>

<hibernate-configuration>

<session-factory>

<property name="connection.driver_class">
com.mysql.cj.jdbc.Driver
</property>

<property name="connection.url">
jdbc:mysql://localhost:3306/test
</property>

<property name="connection.username">
root
</property>

<property name="connection.password">
root
</property>

<property name="dialect">
org.hibernate.dialect.MySQLDialect
</property>

<mapping class="Student"/>

</session-factory>

</hibernate-configuration>
```

Important Configuration Properties

Property	Purpose
connection.driver_class	Database driver class
connection.url	Database URL
connection.username	Database username
connection.password	Database password
dialect	Database-specific SQL dialect
mapping	Registers entity class

Advantages

1. Centralized configuration.
2. Easy database management.
3. Supports different databases.
4. Simplifies SessionFactory creation.
5. Easy maintenance and modification.

Hibernate Configuration Using Annotations

Introduction

Hibernate Annotations are used to configure mapping information directly within Java classes. They eliminate the need for separate XML mapping files and make configuration easier and more readable.

Annotations are provided by JPA (Java Persistence API) and Hibernate.

Common Hibernate Annotations

Annotation	Purpose
@Entity	Marks a class as an entity
@Table	Maps class to database table
@Id	Specifies primary key
@Column	Maps field to column
@GeneratedValue	Generates primary key value automatically

Example

```
import jakarta.persistence.*;

@Entity
@Table(name="student")

public class Student
{
  @Id
```

```
  @GeneratedValue

  private int id;

  @Column(name="name")
  private String name;
}
```

Advantages

1. No separate mapping file required.
2. Easy to understand.
3. Reduces configuration complexity.
4. Improves maintainability.
5. Supports object-oriented development.

Basic CRUD Operations in Hibernate

Introduction

CRUD stands for Create, Read, Update, and Delete. Hibernate provides methods to perform these database operations using Java objects instead of SQL queries.

Create Operation

Used to insert a new record into the database.

Example

```
Session session =
sf.openSession();

Transaction tx =
session.beginTransaction();

session.save(student);

tx.commit();
```

Read Operation

Used to retrieve records from the database.

Example

```
Student s =
session.get(
Student.class,1);
```

Update Operation

Used to modify existing records.

Example

```
Student s =
session.get(
Student.class,1);

s.setName("Anik");

session.update(s);
```

Delete Operation

Used to remove records from the database.

Example

```
Student s =
session.get(
Student.class,1);

session.delete(s);
```

CRUD Methods in Hibernate

Method	Operation
save()	Insert record
get()	Retrieve record
update()	Modify record
delete()	Remove record

Advantages of CRUD in Hibernate

1. Less SQL coding.
2. Easy database interaction.
3. Object-oriented approach.
4. Improved maintainability.
5. Faster application development.

Hibernate Query Language (HQL)

Introduction

Hibernate Query Language (HQL) is an object-oriented query language provided by Hibernate. It is used to retrieve and manipulate data from the database using Java class names and object properties instead of table names and column names.

HQL is similar to SQL but works with Hibernate entities.

Features of HQL

1. Object-oriented query language.
2. Uses class names instead of table names.
3. Uses object properties instead of column names.
4. Database independent.
5. Supports SELECT, INSERT, UPDATE, and DELETE operations.
6. Easy integration with Hibernate.

Basic HQL Syntax

Select Query

String hql =

"FROM Student";

Select Specific Fields

String hql =

"SELECT name FROM Student";

Conditional Query

String hql =

"FROM Student

WHERE id=1";

Update Query

String hql =

"UPDATE Student

SET name='Anik'

WHERE id=1";

Delete Query

String hql =

"DELETE FROM Student

WHERE id=1";

Advantages

1. Easy to write.
2. Database independent.
3. Reduces SQL complexity.
4. Supports object-oriented programming.
5. Improves code portability.

Difference Between SQL and HQL

Introduction

SQL (Structured Query Language) is used to interact directly with relational databases, whereas HQL (Hibernate Query Language) is used in Hibernate to work with Java objects and entities.

Subject	**SQL**	**HQL**
Full Form	Structured Query Language	Hibernate Query Language
Works On	Tables and Columns	Classes and Objects
Database Dependency	Database Specific	Database Independent
Query Target	Database Records	Persistent Objects
Uses Table Names	Yes	No
Uses Class Names	No	Yes
Uses Column Names	Yes	No
Uses Object Properties	No	Yes
Portability	Lower	Higher
Framework Requirement	Not Required	Requires Hibernate

Example

SQL

SELECT * FROM student;

HQL

FROM Student

Advantages of HQL Over SQL

1. Object-oriented approach.
2. Database independence.
3. Better portability.
4. Less database-specific coding.

5. Easy integration with Hibernate applications.